



PÓS-GRADUAÇÃO EM ARQUITETURA E DESENVOLVIMENTO JAVA

Felipe Matos de Lima

RM: 362291

RELATÓRIO TÉCNICO

BOOKING BELEZA — SISTEMA DE AGENDAMENTO PARA SERVIÇOS DE BELEZA E BEM-ESTAR

*Projeto apresentado como requisito de avaliação da fase 3 - Substitutiva, da Pós-Graduação
em Arquitetura e Desenvolvimento Java.*

São Paulo
2026

Sumário

1 INTRODUÇÃO	3
2 TECNOLOGIAS UTILIZADAS.....	4
2.1 Linguagem e Framework.....	4
2.2 Persistência de Dados	4
2.3 Segurança e Autenticação	4
2.4 Documentação da API.....	5
2.5 Notificações	5
2.6 Containerização	5
2.7 Ferramentas de Qualidade de Código	5
2.8 Integração Contínua e Hospedagem	5
3 ARQUITETURA DO SISTEMA	7
3.1 Camada de Domínio (domain).....	7
3.2 Camada de Aplicação (application)	7
3.3 Camada de Infraestrutura (infrastructure)	7
4 FUNCIONALIDADES DO SISTEMA	9
4.1 Cadastro de Estabelecimentos	9
4.2 Perfil de Profissionais.....	9
4.3 Agendamento de Serviços	9
4.4 Avaliações.....	9
4.5 Busca e Filtragem	9
4.6 Gerenciamento de Agendamentos.....	10
4.7 Lembretes Automáticos	10
4.8 Integração com Calendários	10
5 ESTRATÉGIA DE TESTES	11
5.1 Testes Unitários	11
5.2 Testes de Integração	11
5.3 Testes de Comportamento (BDD)	11
5.4 Análise Estática de Código.....	11
5.5 Testes de Performance e Carga.....	12
5.6 Cobertura de Testes	12
5.7 Integração Contínua.....	12
6 DOCUMENTAÇÃO DA API.....	14
7 AMBIENTES DE EXECUÇÃO.....	22
7.1 Ambiente Local.....	22
7.2 Ambiente de Produção	22
8 CONSIDERAÇÕES FINAIS.....	25



1 INTRODUÇÃO

Este documento apresenta o relatório técnico do sistema Booking Beleza, uma aplicação de agendamento desenvolvida para estabelecimentos de beleza e bem-estar. O sistema foi construído em Java 21 com o framework Spring Boot 3, aplicando os princípios de Clean Architecture, SOLID e desenvolvimento orientado a testes, com o objetivo de oferecer uma plataforma robusta para o cadastro de estabelecimentos, gerenciamento de profissionais, agendamento de serviços e acompanhamento pós-atendimento.

O documento descreve as tecnologias empregadas, a arquitetura de software adotada, as funcionalidades implementadas, a estratégia de testes utilizada em cada camada do sistema e os ambientes em que a aplicação está hospedada.

2 TECNOLOGIAS UTILIZADAS

O código-fonte completo do projeto está disponível publicamente no repositório abaixo:

<https://github.com/FMTSL/TECH-CHALLENGE-3>

2.1 Linguagem e Framework

O sistema foi desenvolvido em Java 21, utilizando o framework Spring Boot 3 como base para a construção da aplicação. O Spring Boot fornece a infraestrutura de injeção de dependências, configuração automática e os módulos utilizados no projeto:

- Spring Web — responsável pela exposição dos endpoints REST da API.
- Spring Data JPA — camada de acesso a dados, integrada ao banco de dados relacional por meio do Hibernate.
- Spring Security — responsável pela autenticação e autorização dos usuários da aplicação.
- Spring Mail — utilizado para o envio de notificações por e-mail.
- Spring Validation — validação dos dados recebidos pela API.

2.2 Persistência de Dados

O banco de dados utilizado é o PostgreSQL, acessado por meio do Spring Data JPA e do Hibernate como provedor de ORM (Object-Relational Mapping). O versionamento e a aplicação das alterações de schema do banco de dados são controlados pela ferramenta Flyway, que garante que a estrutura do banco seja criada e atualizada de forma automatizada e auditável a cada execução da aplicação.

2.3 Segurança e Autenticação

A autenticação dos usuários é realizada por meio de tokens JWT (JSON Web Token), implementados com a biblioteca JJWT. O sistema utiliza autenticação stateless, na qual cada requisição autenticada carrega um token assinado que identifica o usuário e seu perfil de acesso, dispensando o uso de sessões no servidor.

2.4 Documentação da API

A documentação interativa da API é gerada automaticamente por meio da biblioteca `springdoc-openapi`, que expõe uma interface Swagger UI com a relação completa de endpoints, parâmetros e modelos de dados da aplicação. Complementarmente, as classes do sistema possuem documentação Javadoc descrevendo a responsabilidade de cada componente.

2.5 Notificações

As notificações de confirmação, cancelamento e lembrete de agendamento são enviadas por e-mail por meio do Spring Mail. No ambiente de desenvolvimento, essas mensagens são interceptadas e podem ser visualizadas na interface do MailHog, uma ferramenta que simula um servidor de e-mail para fins de teste, disponível em <http://localhost:8025>.

2.6 Containerização

A aplicação é empacotada em uma imagem Docker, construída a partir de um processo de build em múltiplos estágios (multi-stage build), que compila o código-fonte com Maven e gera uma imagem final otimizada para execução. O Docker Compose é utilizado para orquestrar, em ambiente local, os contêineres da aplicação, do banco de dados PostgreSQL e do MailHog.

2.7 Ferramentas de Qualidade de Código

- Checkstyle — verificação de convenções e padrões de estilo de código.
- SpotBugs — identificação de padrões de código com potencial de causar defeitos.
- PMD — análise de complexidade e boas práticas de design de código.
- JaCoCo — geração de relatórios de cobertura de testes.

2.8 Integração Contínua e Hospedagem

- GitHub Actions — automação do processo de build, testes e implantação da aplicação.
- Amazon Web Services (AWS) — hospedagem da aplicação em contêineres, por meio do serviço Elastic Container Service (ECS) com o modelo de execução Fargate.



- Render — hospedagem do banco de dados PostgreSQL gerenciado, consumido pela aplicação em execução na AWS.

3 ARQUITETURA DO SISTEMA

O sistema foi estruturado seguindo os princípios da Clean Architecture, organizando o código em camadas concêntricas com uma regra de dependência unidirecional: as camadas mais externas dependem das mais internas, e nunca o contrário. Essa organização isola as regras de negócio de detalhes de implementação, como o framework web utilizado ou o mecanismo de persistência de dados.

3.1 Camada de Domínio (domain)

Contém as entidades de negócio e as regras que lhes são inerentes, implementadas como classes Java simples (POJOs), sem qualquer anotação ou dependência de frameworks externos. Também reúne as interfaces de repositório, que definem os contratos de persistência sem conhecer sua implementação concreta, e as exceções de domínio.

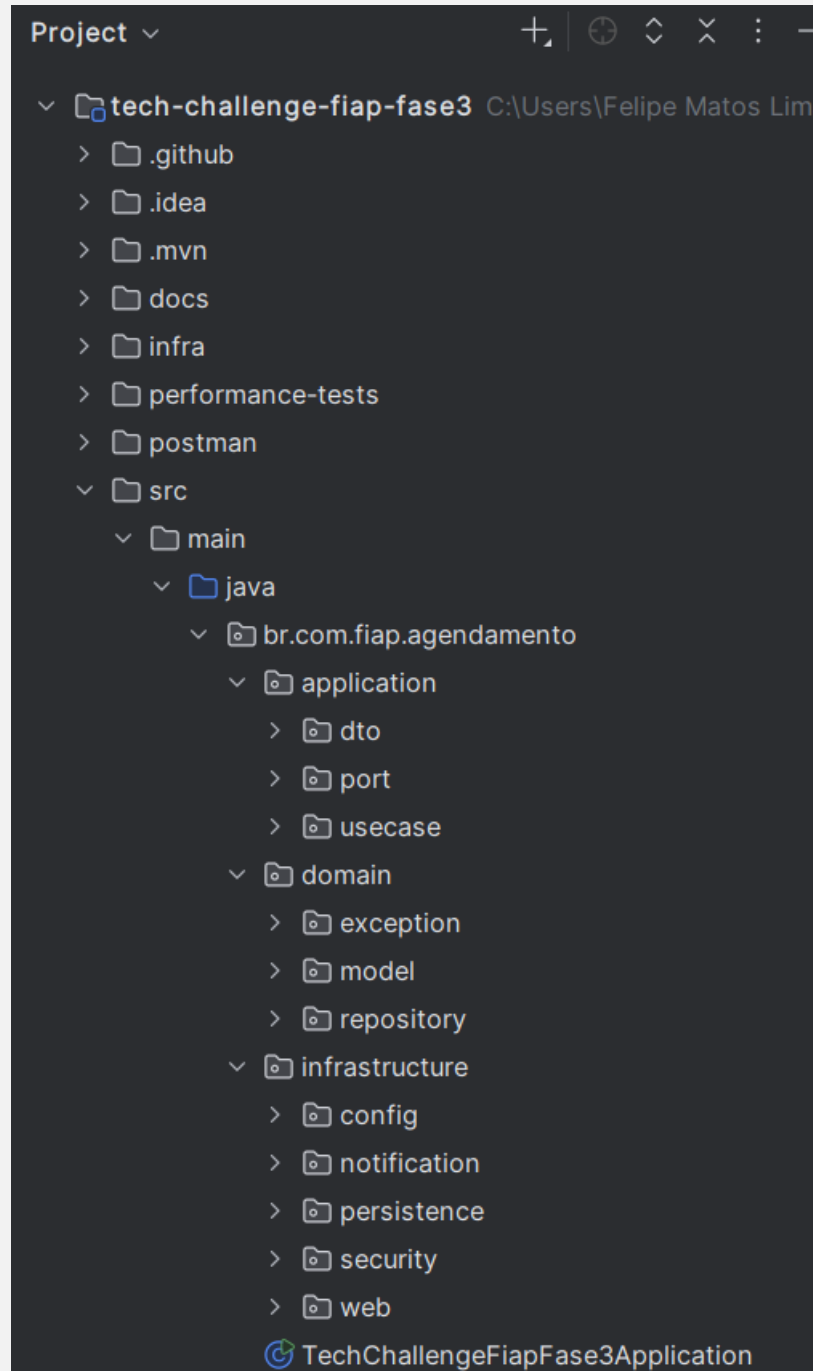
3.2 Camada de Aplicação (application)

Reúne os casos de uso do sistema, cada um responsável por orquestrar uma única operação de negócio, seguindo o princípio da responsabilidade única. Os casos de uso dependem exclusivamente das interfaces definidas na camada de domínio, nunca de implementações concretas, em conformidade com o princípio de inversão de dependência.

3.3 Camada de Infraestrutura (infrastructure)

Contém os detalhes de implementação e a integração com tecnologias externas: os controladores REST, a configuração de segurança e autenticação, o envio de notificações e a camada de persistência de dados. Esta última é organizada em quatro subcamadas — entidades JPA, mapeadores entre entidades de domínio e de persistência, repositórios Spring Data e adaptadores que implementam os contratos definidos na camada de domínio — de modo que o modelo de domínio permanece completamente independente da tecnologia de persistência utilizada.

FIAP + POSTECH



4 FUNCIONALIDADES DO SISTEMA

O sistema disponibiliza as seguintes funcionalidades, acessíveis por meio de uma API REST documentada:

4.1 Cadastro de Estabelecimentos

Permite que o usuário responsável por um estabelecimento realize seu cadastro, informando nome, endereço, cidade, horário de funcionamento e fotos.

4.2 Perfil de Profissionais

Cada profissional vinculado a um estabelecimento possui um perfil com suas especialidades, tarifa base e uma agenda semanal recorrente, composta por janelas de horário disponível em cada dia da semana. A partir dessa agenda, o sistema calcula automaticamente os horários livres para agendamento em uma data específica.

4.3 Agendamento de Serviços

O cliente seleciona um profissional, um serviço e um horário disponível para realizar o agendamento. O sistema aplica duas camadas de proteção contra agendamentos conflitantes no mesmo horário para o mesmo profissional: uma validação em memória, realizada no momento da requisição, e uma restrição de unicidade aplicada diretamente no banco de dados, que garante a integridade da informação mesmo em cenários de requisições concorrentes. Após a confirmação do agendamento, o sistema envia uma notificação por e-mail ao cliente e ao profissional.

4.4 Avaliações

Após a conclusão de um atendimento, o cliente pode registrar uma avaliação composta por uma nota, de um a cinco, e um comentário. A nota média das avaliações recebidas por um estabelecimento é exibida nos resultados de busca.

4.5 Busca e Filtragem

Os estabelecimentos podem ser localizados por nome ou cidade, com filtragem adicional por serviço oferecido, faixa de preço, nota mínima de avaliação e disponibilidade de horário em uma data específica.

4.6 Gerenciamento de Agendamentos

O estabelecimento possui um painel para acompanhar os agendamentos recebidos e atualizar seu status — confirmado, concluído ou não comparecido. O cliente pode cancelar ou reagendar um agendamento existente, sujeito às mesmas regras de validação de disponibilidade aplicadas na criação.

4.7 Lembretes Automáticos

Um processo agendado, executado periodicamente pela aplicação, identifica os agendamentos confirmados que ocorrerão nas próximas vinte e quatro horas e envia um lembrete por e-mail ao cliente e ao profissional envolvidos.

4.8 Integração com Calendários

O sistema disponibiliza um recurso de sincronização de agendamentos com calendários externos, por meio da geração de um arquivo no formato iCalendar (.ics), compatível com aplicativos como Google Calendar, Outlook e Apple Calendar. O cliente autenticado pode obter o feed de seus próprios agendamentos, e cada profissional possui um feed próprio, acessível por meio de um endereço de assinatura público.

5 ESTRATÉGIA DE TESTES

A qualidade do sistema foi assegurada por meio de múltiplas camadas de teste, cada uma responsável por validar um aspecto distinto da aplicação, descritas a seguir.

5.1 Testes Unitários

Os testes unitários validam cada caso de uso da camada de aplicação de forma isolada, utilizando o framework JUnit 5 em conjunto com a biblioteca Mockito para a criação de dublês de teste (mocks) das dependências externas, e a biblioteca AssertJ para a escrita de asserções. Como os casos de uso dependem exclusivamente de interfaces, é possível testá-los sem a necessidade de um banco de dados ou de qualquer infraestrutura externa, cobrindo o comportamento esperado em cenários de sucesso, exceções de negócio e casos de borda — como a regra de concorrência que impede o agendamento duplo de um mesmo horário.

5.2 Testes de Integração

Os testes de integração validam o funcionamento conjunto de diferentes componentes do sistema, exercitando o fluxo completo de uma requisição — do controlador REST até a persistência no banco de dados. Esses testes utilizam a biblioteca Testcontainers, que inicializa uma instância real do banco de dados PostgreSQL em um contêiner Docker durante a execução dos testes, garantindo que o comportamento validado corresponda ao ambiente de produção.

5.3 Testes de Comportamento (BDD)

Complementando os testes de integração, o sistema adota a prática de Behaviour-Driven Development (BDD) por meio do framework Cucumber. Os cenários de teste são escritos em linguagem natural, no idioma português, utilizando a notação Gherkin, o que aproxima a especificação técnica da linguagem de negócio e facilita a comunicação entre desenvolvedores e demais interessados no projeto. Os cenários cobrem o fluxo de agendamento de um serviço, a rejeição de um agendamento duplicado no mesmo horário e o cancelamento de um agendamento.

5.4 Análise Estática de Código

A qualidade estrutural do código-fonte é verificada por três ferramentas complementares, executadas automaticamente durante o processo de build: o Checkstyle verifica a aderência a convenções de estilo e formatação; o SpotBugs identifica padrões de código associados a defeitos conhecidos; e o PMD avalia a complexidade e as boas práticas de design das classes e métodos.

5.5 Testes de Performance e Carga

Os testes não funcionais de performance são realizados com a ferramenta k6, que simula um volume crescente de usuários realizando requisições simultâneas à aplicação. O cenário de teste eleva gradualmente a carga até cem usuários virtuais simultâneos, validando que o tempo de resposta e a taxa de erro da aplicação permanecem dentro de limites aceitáveis mesmo sob alta concorrência.

5.6 Cobertura de Testes

A cobertura dos testes automatizados é medida pela ferramenta JaCoCo, integrada ao processo de build do projeto, que gera um relatório detalhado indicando o percentual de classes, métodos e linhas de código exercitados pela suíte de testes.

5.7 Integração Contínua

Todo o processo de testes é automatizado por meio de um pipeline de integração contínua configurado no GitHub Actions, executado a cada alteração enviada ao repositório. O pipeline realiza a compilação do projeto, a execução dos testes unitários, dos testes de integração, da análise estática de código e a verificação da construção da imagem Docker da aplicação, assegurando que apenas código validado seja incorporado ao projeto.

FIAP + POSTECH

Upcoming change to GitHub App installation token format

GitHub App installation tokens will soon use a new stateless format (ghs_...) and may be longer (~520 characters). Apps with hardcoded length assumptions may break. Validate your apps and workflows with the per-request override header detailed in this [GitHub Changelog](#).

All workflows

Showing runs from all workflows

Filter workflow runs

Workflow	Event	Status	Branch	Actor
fix: correção da busca estabelecimentos	CI R25: Commit cbaf6d6 pushed by fmltsl	pass	main	Jul 26, 00:56 GMT-3 3m 14s
fix: ajuste na collections do postman para produção	CI R24: Commit 3bc086c pushed by fmltsl	pass	main	Jul 25, 13:18 GMT-3 3m 19s
fix: primeiros ajustes de documentação do projeto	CI R23: Commit 3d59f6d pushed by fmltsl	pass	main	Jul 24, 01:50 GMT-3 2m 48s
ajuste no deploy aws	CI R22: Commit 4b6d077 pushed by fmltsl	pass	main	Jul 24, 01:31 GMT-3 3m 37s
fix: otimiza hikari e ssl para boot rapido na Render	CI R21: Commit af8c0d7 pushed by fmltsl	pass	main	Jul 23, 14:10 GMT-3 3m 5s
fix: otimiza boot do Java e aplica JAVA_OPTS no ENTRYPOINT	CI R20: Commit 1a67c1d pushed by fmltsl	pass	main	Jul 23, 13:41 GMT-3 2m 40s

6 DOCUMENTAÇÃO DA API

A API do sistema é documentada de forma interativa por meio do Swagger UI, que apresenta a relação completa de endpoints disponíveis, seus parâmetros de entrada, os modelos de dados esperados e retornados, e permite a execução de requisições diretamente pela interface. A documentação está disponível no endereço:

Local: <http://localhost:8080/swagger-ui/index.html>

Produção: <http://18.231.160.214:8080/swagger-ui/index.html>

Complementarmente, as classes do sistema — entidades de domínio, casos de uso, controladores e demais componentes — possuem documentação Javadoc descrevendo sua responsabilidade e, quando pertinente, as decisões de design adotadas.

Testando a API:

O diretório “postman/” contém duas coleções do Postman com os mesmos 26 testes manuais dos fluxos principais do sistema — “booking-beleza.postman_collection.json” (local) e “booking-beleza-producao.postman_collection.json” (ambiente de produção na AWS) — prontas para importar e executar. Token e IDs (estabelecimento, profissional, serviço, agendamento) são capturados automaticamente entre requisições via scripts, sem necessidade de copiar valores manualmente.

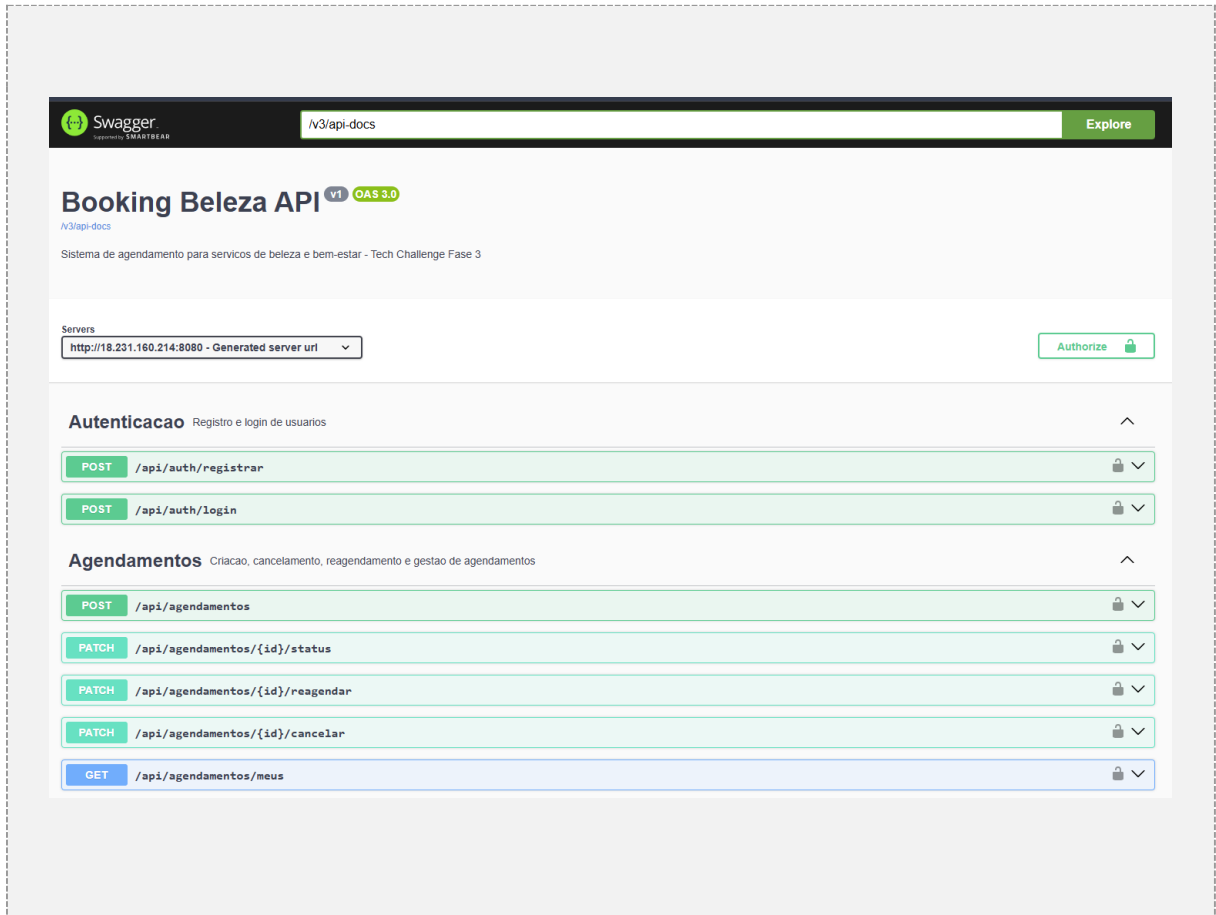
O documento “postman/README.md” descreve a mesma sequência de testes de duas formas equivalentes:

- **Via Postman:** importação da coleção e execução em lote (Run collection) ou requisição por requisição.
- **Via Swagger UI:** os mesmos passos reproduzidos manualmente em “/swagger-ui.html”, com os corpos de requisição prontos para copiar.

A sequência cobre: registro e login de cliente e estabelecimento; cadastro de estabelecimento, profissional e serviço; definição de disponibilidade; criação de agendamento (incluindo o teste da regra de double-booking, que deve retornar `409` na segunda tentativa no mesmo horário);

FIAP + POSTECH

confirmação, reagendamento e conclusão de um agendamento; avaliação; feeds de calendário do cliente e do profissional; e os filtros de busca avançada.



The image shows the Swagger UI for the 'Booking Beleza API'. The interface is clean and modern, with a dark header bar containing the Swagger logo and the API name. Below the header, the API title 'Booking Beleza API' is displayed with a version indicator 'v1' and an 'OAS 3.0' badge. A description of the API is provided: 'Sistema de agendamento para serviços de beleza e bem-estar - Tech Challenge Fase 3'. The 'Servers' section shows a single server URL: 'http://18.231.160.214:8080 - Generated server url'. The 'Authorize' button is visible. The API endpoints are organized into two main sections: 'Autenticacao' (Authentication) and 'Agendamentos' (Bookings). The 'Autenticacao' section includes two endpoints: 'POST /api/auth/registrar' and 'POST /api/auth/login'. The 'Agendamentos' section includes five endpoints: 'POST /api/agendamentos', 'PATCH /api/agendamentos/{id}/status', 'PATCH /api/agendamentos/{id}/reagendar', 'PATCH /api/agendamentos/{id}/cancelar', and 'GET /api/agendamentos/meus'. Each endpoint is represented by a colored bar with the HTTP method, the path, and a lock icon indicating security requirements.

Swagger
OpenAPI Specification

/v3/api-docs Explore

Booking Beleza API v1 OAS 3.0

/v3/api-docs

Sistema de agendamento para serviços de beleza e bem-estar - Tech Challenge Fase 3

Servers

http://18.231.160.214:8080 - Generated server url

Authorize

Autenticacao Registro e login de usuarios

- POST /api/auth/registrar
- POST /api/auth/login

Agendamentos Criacao, cancelamento, reagendamento e gestao de agendamentos

- POST /api/agendamentos
- PATCH /api/agendamentos/{id}/status
- PATCH /api/agendamentos/{id}/reagendar
- PATCH /api/agendamentos/{id}/cancelar
- GET /api/agendamentos/meus

FIAP + POSTECH

GET	/api/agendamentos/estabelecimento/{estabelecimentoId}	🔒 ▼
Disponibilidade Agenda semanal e horarios livres dos profissionais ^		
GET	/api/profissionais/{profissionalId}/disponibilidades	🔒 ▼
POST	/api/profissionais/{profissionalId}/disponibilidades	🔒 ▼
GET	/api/profissionais/{profissionalId}/disponibilidades/slots-livres	🔒 ▼
Profissionais Perfis de profissionais e suas especialidades ^		
GET	/api/profissionais	🔒 ▼
POST	/api/profissionais	🔒 ▼
Avaliacoes Avaliacoes e comentarios pos-atendimento ^		
POST	/api/avaliacoes	🔒 ▼
Calendario Feed ICalendar para integracao com Google Calendar/Outlook/Apple Calendar ^		
GET	/api/calendario/profissionais/{profissionalId}/feed.ics	🔒 ▼
GET	/api/calendario/meu-feed.ics	🔒 ▼
Estabelecimentos Cadastro, busca e filtragem avancada de estabelecimentos ^		
GET	/api/estabelecimentos	🔒 ▼
POST	/api/estabelecimentos	🔒 ▼
GET	/api/estabelecimentos/{id}	🔒 ▼
Servicos Catalogo de servicos oferecidos pelos estabelecimentos ^		
GET	/api/servicos	🔒 ▼
POST	/api/servicos	🔒 ▼

FIAP + POSTECH

← → ↺ 🏠 ⓘ Arquivo C:/Users/Felipe%20Matos%20Lima/Downloads/tech-challenge-fiap-fase3/tech-challenge-fiap-fase3/target/site/apidocs/index.html	
OVERVIEW PACKAGE CLASS USE TREE INDEX HELP	
tech-challenge-fiap-fase3 0.0.1-SNAPSHOT API	
Packages	
Package	Description
br.com.fiap.agendamento	
br.com.fiap.agendamento.application.dto	
br.com.fiap.agendamento.application.port	
br.com.fiap.agendamento.application.usecase.agendamento	
br.com.fiap.agendamento.application.usecase.auth	
br.com.fiap.agendamento.application.usecase.avaliacao	
br.com.fiap.agendamento.application.usecase.calendario	
br.com.fiap.agendamento.application.usecase.disponibilidade	
br.com.fiap.agendamento.application.usecase.estabelecimento	
br.com.fiap.agendamento.application.usecase.profissional	
br.com.fiap.agendamento.application.usecase.servico	
br.com.fiap.agendamento.domain.exception	
br.com.fiap.agendamento.domain.model	
br.com.fiap.agendamento.domain.repository	
br.com.fiap.agendamento.infrastructure.config	
br.com.fiap.agendamento.infrastructure.notification	
br.com.fiap.agendamento.infrastructure.persistence.adapter	
br.com.fiap.agendamento.infrastructure.persistence.entity	
br.com.fiap.agendamento.infrastructure.persistence.mapper	
br.com.fiap.agendamento.infrastructure.persistence.springdata	
br.com.fiap.agendamento.infrastructure.security	
br.com.fiap.agendamento.infrastructure.web	
br.com.fiap.agendamento.infrastructure.web.controller	
br.com.fiap.agendamento.infrastructure.web.exception	
Copyright © 2026. All rights reserved.	

FIAP + POSTECH

Booking Beleza - Tech C

No environment

Booking Beleza - Tech Chall... - Run results

Run Again + New Run Automate Run Share

Ran today at 09:05:53 PM · View all runs

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	5s 980ms	13	0	109 ms

All Tests

Passed (13) Failed (0) Skipped (0) Errors (0) Console Log

View Summary

Iteration 1

1

POST 1. Autenticacao / 1.1 Registrar Cliente

http://localhost:8080/api/auth/regar

201 • 856 ms • 387 B • 1

PASS Status 201

POST 1. Autenticacao / 1.2 Login Cliente

http://localhost:8080/api/auth/login

200 • 291 ms • 723 B • 1

PASS Status 200

POST 1. Autenticacao / 1.3 Registrar Dono do Estabelecimento

http://localhost:8080/api/auth/regar

201 • 87 ms • 387 B • 1

PASS Status 201

POST 1. Autenticacao / 1.4 Login Dono

http://localhost:8080/api/auth/login

200 • 87 ms • 729 B • 1

PASS Status 200

POST 2. Cadastro (Estabelecimento, Profissional, Servico) / 2.1 Cadastrar Estabelecimento

http://localhost:8080/api/estabelecimentos

201 • 165 ms • 639 B • 1

PASS Status 201

POST 2. Cadastro (Estabelecimento, Profissional, Servico) / 2.2 Cadastrar Profissional

http://localhost:8080/api/profissionais

201 • 425 ms • 608 B • 1

PASS Status 201

POST 2. Cadastro (Estabelecimento, Profissional, Servico) / 2.3 Cadastrar Servico

http://localhost:8080/api/servicos

201 • 53 ms • 618 B • 1

PASS Status 201

FIAP + POSTECH

Booking Beleza - Tech C

No environment

Booking Beleza - Tech Chall... - Run results

Run Again

New Run

Automate Run

Share

Ran today at 09:05:53 PM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	5s 980ms	13	0	109 ms

All Tests Passed (13) Failed (0) Skipped (0) Errors (0) Console Log [View Summary](#)

GET

2. Cadastro (Estabelecimento, Profissional, Servico) / 2.4 Listar Estabelecimentos (busca)

http://localhost:8080/api/estabelecimentos?q=Sao Paulo

200

56 ms

636 B

No tests found

GET

2. Cadastro (Estabelecimento, Profissional, Servico) / 2.5 Buscar Estabelecimento por ID

http://localhost:8080/api/estabelecimentos/e90400ec-fe98-48ab-b44c-508d6913845b

200

27 ms

634 B

No tests found

POST

3. Disponibilidade do Profissional / 3.1 Definir Disponibilidade

http://localhost:8080/api/profissionais/27634a21-7da0-4565-b345-a272581c240e/disponibilidades

201

44 ms

538 B

1

PASS Status 201

GET

3. Disponibilidade do Profissional / 3.2 Listar Janelas de Disponibilidade

http://localhost:8080/api/profissionais/27634a21-7da0-4565-b345-a272581c240e/disponibilidades

200

14 ms

535 B

No tests found

GET

3. Disponibilidade do Profissional / 3.3 Consultar Slots Livres

http://localhost:8080/api/profissionais/27634a21-7da0-4565-b345-a272581c240e/disponibilidades/slots-livres?data=2026-08-04

200

34 ms

820 B

No tests found

POST

4. Agendamento / 4.1 Criar Agendamento

http://localhost:8080/api/agendamentos

201

244 ms

743 B

1

PASS Status 201

POST

4. Agendamento / 4.2 Tentar Agendar o MESMO horario (deve dar 409)

http://localhost:8080/api/agendamentos

409

44 ms

625 B

1

PASS Status 409 (double-booking bloqueado)

GET

4. Agendamento / 4.3 Meus Agendamentos (Cliente)

http://localhost:8080/api/agendamentos/meus

200

31 ms

740 B

FIAP + POSTECH

Booking Beleza - Tech

No environment

Booking Beleza - Tech Chall... - Run results

Run Again + New Run Automate Run Share

Ran today at 09:05:53 PM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	5s 980ms	13	0	109 ms

All Tests

Passed (13)

Failed (0)

Skipped (0)

Errors (0)

Console Log

View Summary

GET

4. Agendamento / 4.4 Paine! do Estabelecimento

http://localhost:8080/api/agendamentos/estabelecimento/e90400ec-fe98-48ab-b44c-508d6913845b

200

20 ms

740 B

No tests found

PATCH

4. Agendamento / 4.5 Confirmar Agendamento (Estabelecimento)

http://localhost:8080/api/agendamentos/bfd959bf-a6d7-47cf-9520-5e1f10358530/status?status=CONFIRMADO

200

43 ms

740 B

1

PASS Status 200

PATCH

4. Agendamento / 4.6 Reagendar

http://localhost:8080/api/agendamentos/bfd959bf-a6d7-47cf-9520-5e1f10358530/reagendar

409

41 ms

653 B

No tests found

PATCH

4. Agendamento / 4.7 Marcar como Concluido (Estabelecimento)

http://localhost:8080/api/agendamentos/bfd959bf-a6d7-47cf-9520-5e1f10358530/status?status=CONCLUIDO

200

23 ms

739 B

1

PASS Status 200

POST

5. Avaliacao / 5.1 Avaliar Atendimento

http://localhost:8080/api/avaliacoes

201

61 ms

690 B

1

PASS Status 201

GET

6. Calendario (iCal) / 6.1 Feed do Cliente (.ics)

http://localhost:8080/api/calendario/meu-feed.ics

200

19 ms

722 B

No tests found

GET

6. Calendario (iCal) / 6.2 Feed do Profissional (.ics, publico)

http://localhost:8080/api/calendario/profissionais/27634a21-7da0-4565-b345-a272581c240e/feed.ics

200

15 ms

722 B

No tests found

GET

7. Busca Avancada de Estabelecimentos / 7.1 Por servico oferecido

http://localhost:8080/api/estabelecimentos?servico=Corte

200

33 ms

635 B

FIAP + POSTECH

GET 7. Busca Avancada de Estabelecimentos / **7.2 Por faixa de preco**

<http://localhost:8080/api/estabelecimentos?precoMin=50&precoMax=150>

200 • 30 ms • 635 B •

No tests found

GET 7. Busca Avancada de Estabelecimentos / **7.3 Por nota minima**

<http://localhost:8080/api/estabelecimentos?notaMinima=4>

200 • 27 ms • 635 B •

No tests found

GET 7. Busca Avancada de Estabelecimentos / **7.4 Por disponibilidade em uma data**

<http://localhost:8080/api/estabelecimentos?disponivelEm=2026-08-04>

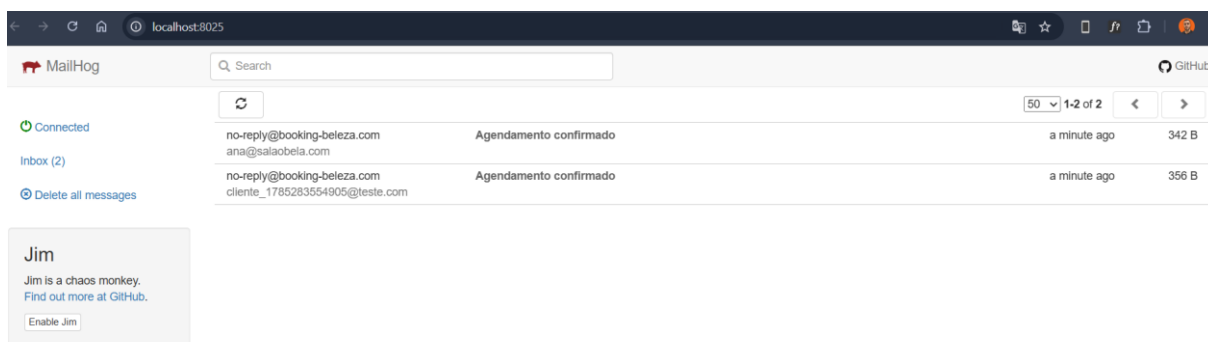
200 • 52 ms • 635 B •

No tests found

7 AMBIENTES DE EXECUÇÃO

7.1 Ambiente Local

Para desenvolvimento e testes locais, a aplicação é executada por meio do Docker Compose, que orquestra três contêineres: a aplicação Spring Boot, o banco de dados PostgreSQL e o MailHog. O MailHog atua como um servidor de e-mail simulado, permitindo a visualização das notificações enviadas pela aplicação — confirmação, cancelamento e lembrete de agendamento — em uma interface web, sem a necessidade de um servidor de e-mail real. A interface do MailHog é acessada em <http://localhost:8025>.



7.2 Ambiente de Produção

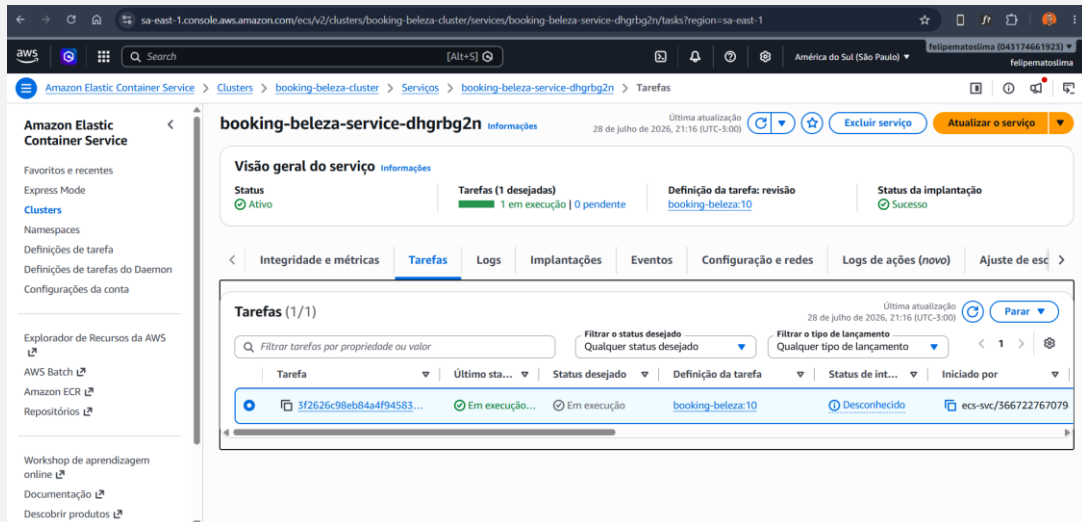
A aplicação encontra-se hospedada na plataforma Amazon Web Services (AWS), em um cluster Elastic Container Service (ECS) com o modelo de execução Fargate, que dispensa o gerenciamento direto de servidores. A aplicação está disponível nos seguintes endereços:

<http://18.231.160.214:8080>

<http://18.231.160.214:8080/swagger-ui/index.html>

O banco de dados PostgreSQL utilizado pela aplicação em produção está hospedado na plataforma Render, como um serviço de banco de dados gerenciado. A aplicação, em execução na AWS, conecta-se a esse banco de dados por meio de uma conexão externa criptografada, estabelecendo uma arquitetura na qual a aplicação e o banco de dados residem em provedores de nuvem distintos.

FIAP + POSTECH



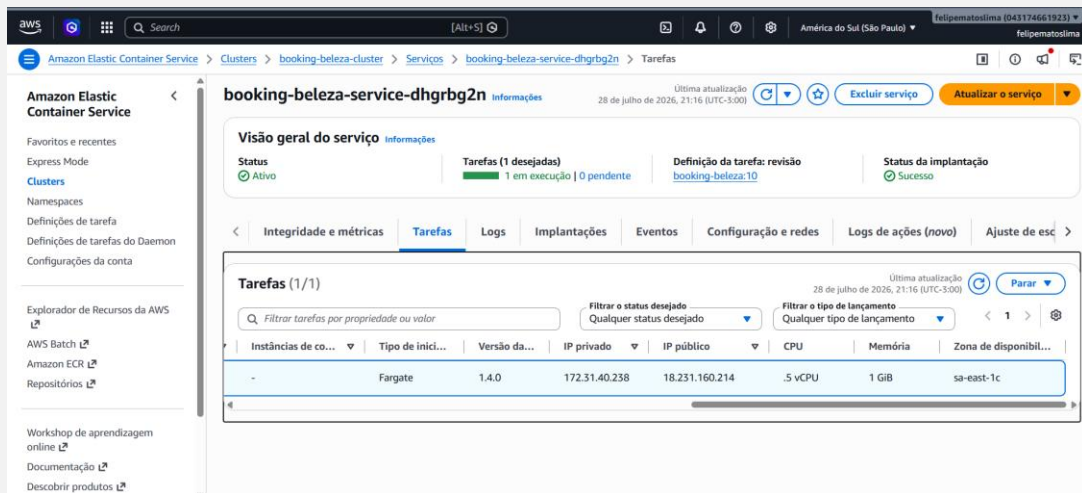
The screenshot shows the AWS Elastic Container Service console for the 'booking-beleza-service-dhgrbg2n' service. The service is active and has 1 task in execution. The task is named 'booking-beleza:10'.

Visão geral do serviço

- Status: Ativo
- Tarefas (1 desejadas): 1 em execução | 0 pendente
- Definição da tarefa: revisão booking-beleza:10
- Status da implantação: Sucesso

Tarefas (1/1)

Tarefa	Último sta...	Status desejado	Definição da tarefa	Status de int...	Iniciado por
3f2626c98eb84a4f94583...		Em execução	booking-beleza:10	Desconhecido	ecs-svc/366722767079



The screenshot shows the AWS Elastic Container Service console for the 'booking-beleza-service-dhgrbg2n' service. The service is active and has 1 task in execution. The task is named 'booking-beleza:10'.

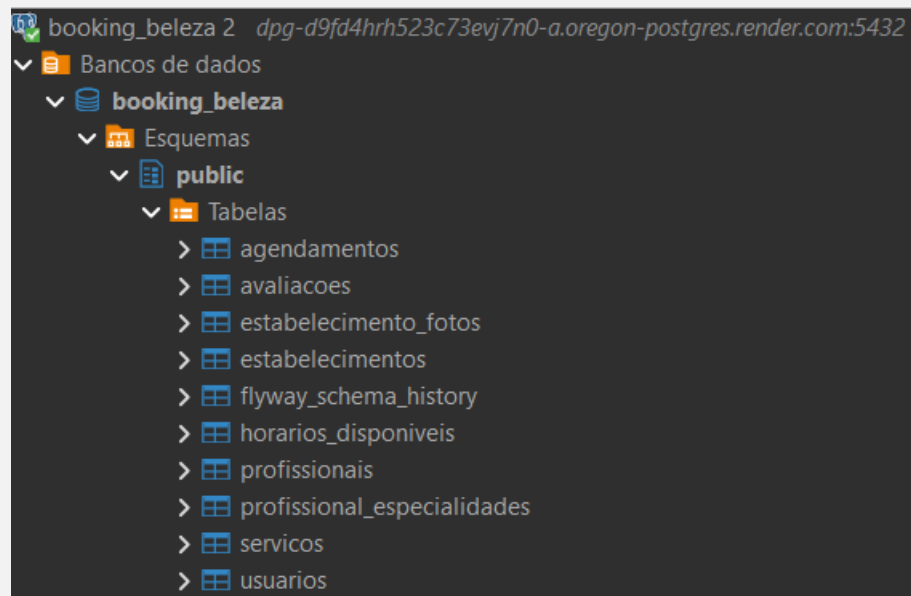
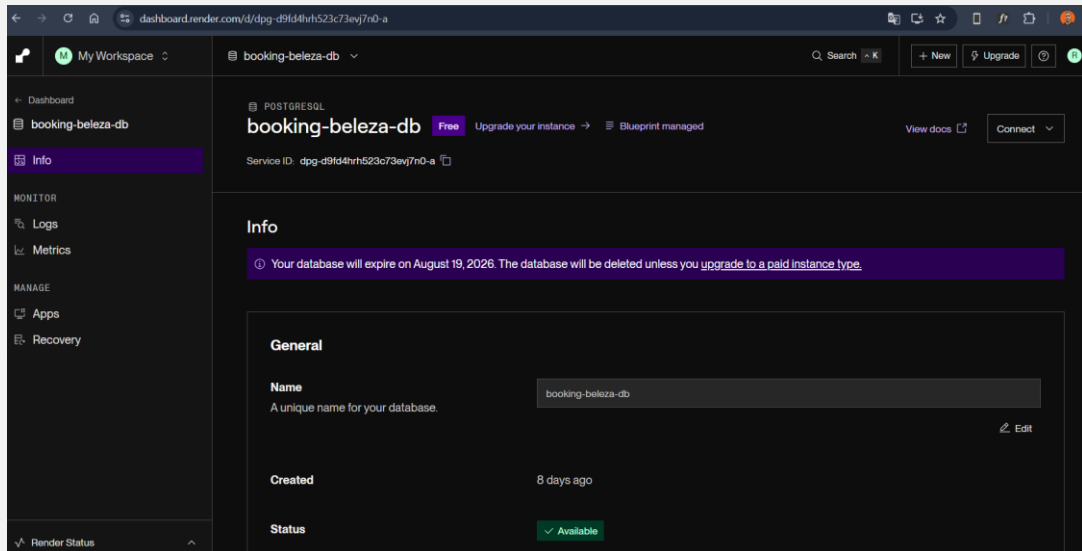
Visão geral do serviço

- Status: Ativo
- Tarefas (1 desejadas): 1 em execução | 0 pendente
- Definição da tarefa: revisão booking-beleza:10
- Status da implantação: Sucesso

Tarefas (1/1)

Instâncias de co...	Tipo de inici...	Versão da...	IP privado	IP público	CPU	Memória	Zona de disponib...
-	Fargate	1.4.0	172.31.40.238	18.231.160.214	.5 vCPU	1 GiB	sa-east-1c

FIAP + POSTECH



8 CONSIDERAÇÕES FINAIS

O sistema Booking Beleza foi desenvolvido em conformidade com os princípios de Clean Architecture, mantendo o modelo de domínio isolado de qualquer dependência de framework, e com práticas consolidadas de qualidade de software, incluindo testes unitários, testes de integração, testes de comportamento, análise estática de código e testes de performance. A aplicação está em funcionamento em ambiente de produção, com a camada de aplicação hospedada na Amazon Web Services e o banco de dados hospedado na plataforma Render, demonstrando a viabilidade de uma arquitetura distribuída entre provedores de nuvem distintos.